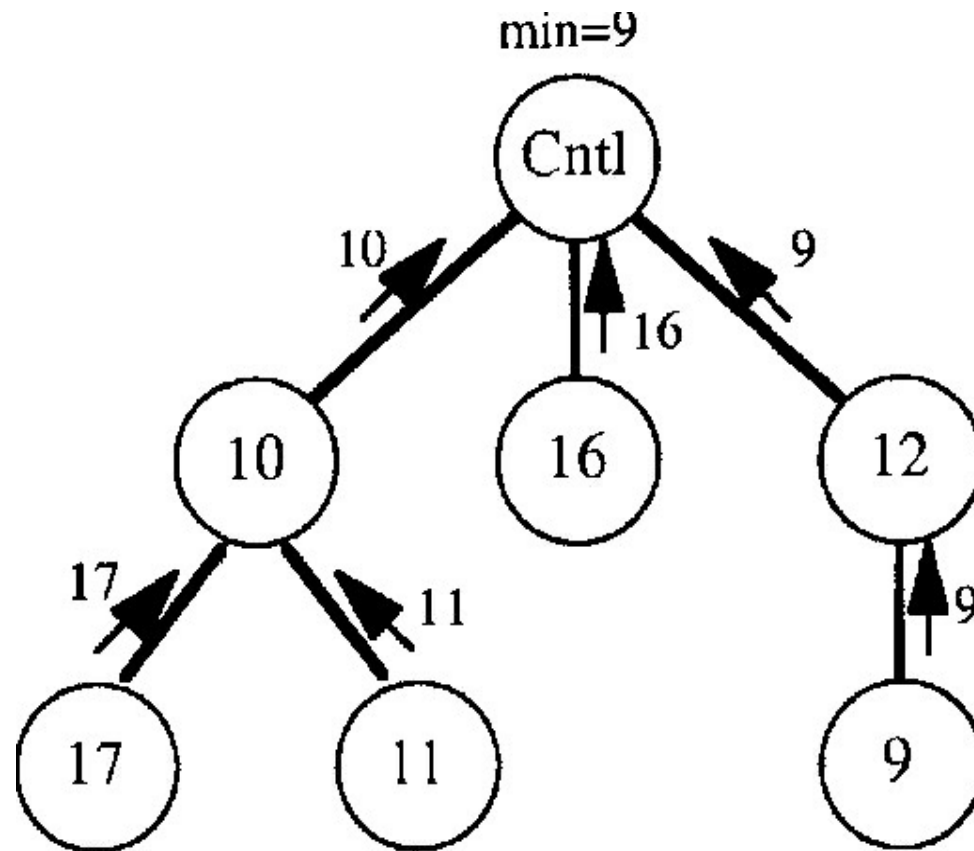


Example deadlock recovery mechanism



Summary

- Deadlock Detection
 - Diffusing computation: Dijkstra/Scholten algorithm
 - Simple signaling protocol detects deadlock
 - Does not detect partial (local) deadlocks
- Deadlock Recovery
 - Smallest time stamp event safe to process
 - Others may also be safe (requires additional work to determine this)

Parallel and Distributed Simulation

Synchronous Execution

Barrier Mechanisms

Performance of Conservative Approaches

Outline

- Synchronous Algorithms
 - Barrier mechanisms
 - Centralized Barriers
 - Tree Barrier
- Transient Message Problem
- Performance of Conservative Approaches

Synchronous Execution

Basic idea: each process cycles through the following steps:

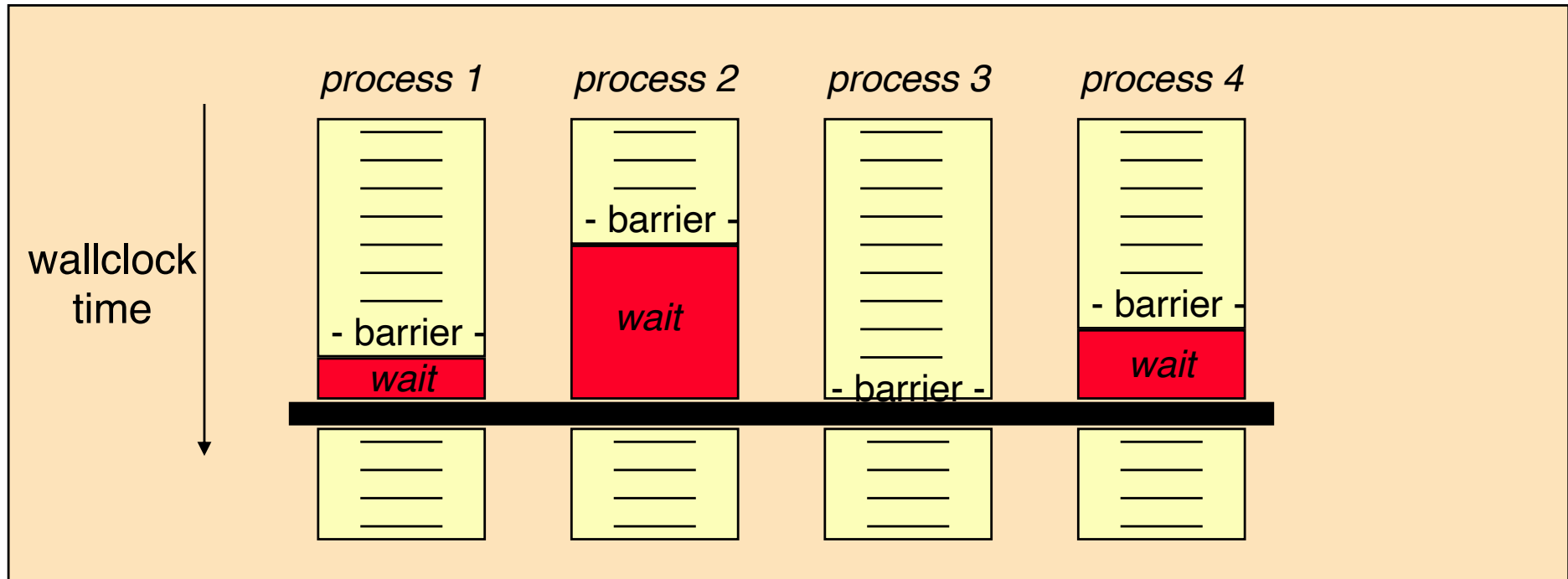
- Determine the events that are safe to process
- Process events, exchange messages
- Global synchronization (barrier)

Messages generated in one cycle are not eligible for processing until the next cycle

Issues

- Barrier mechanism, transient messages
- Determining safe events

Barrier Synchronization



Barrier Synchronization: when a process invokes the barrier primitive, it will block until all other processors have also invoked the barrier primitive.

When the last process invokes the barrier, all processes can execute forward

Barrier Implementation

Centralized Message-Passing Approach

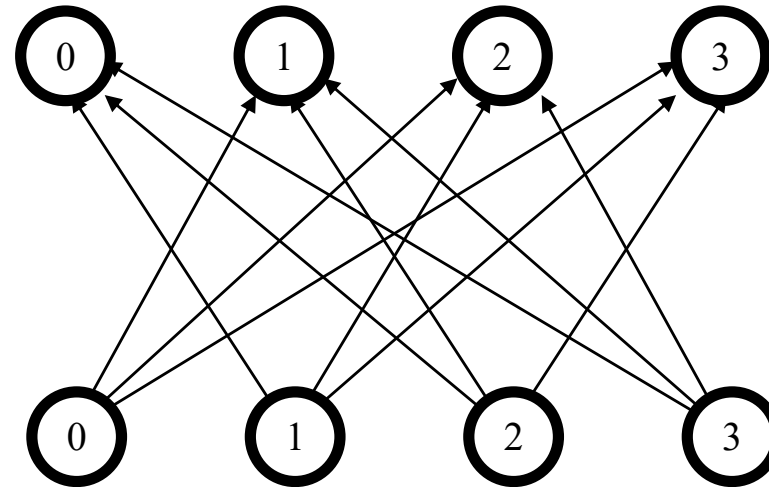
- Central controller used to implement barrier
- 2 step process
 - Determine when barrier reached
 - Broadcast message to release processes from the barrier
- Barrier primitive for non-controller processes:
 - Send a message to central controller
 - Wait for a reply
- Barrier primitive for controller process
 - Receive barrier messages from other processes
 - When a message is received from each process, broadcast message to release barrier
- Performance
 - Controller must send and receive $N-1$ messages
 - Potential bottleneck

Algorithm for Barrier Synchronization

```
WHILE (simulation is not over)
    identify all events that are safe to process
    barrier synchronization
    process safe events
    barrier synchronization
END-LOOP
```

The principal drawback with the centralized approach is that it does **not scale to large numbers of processors**, since the central controller, become a **bottleneck**.
The controller must perform $N-1$ message sends and receives on each barrier operation.

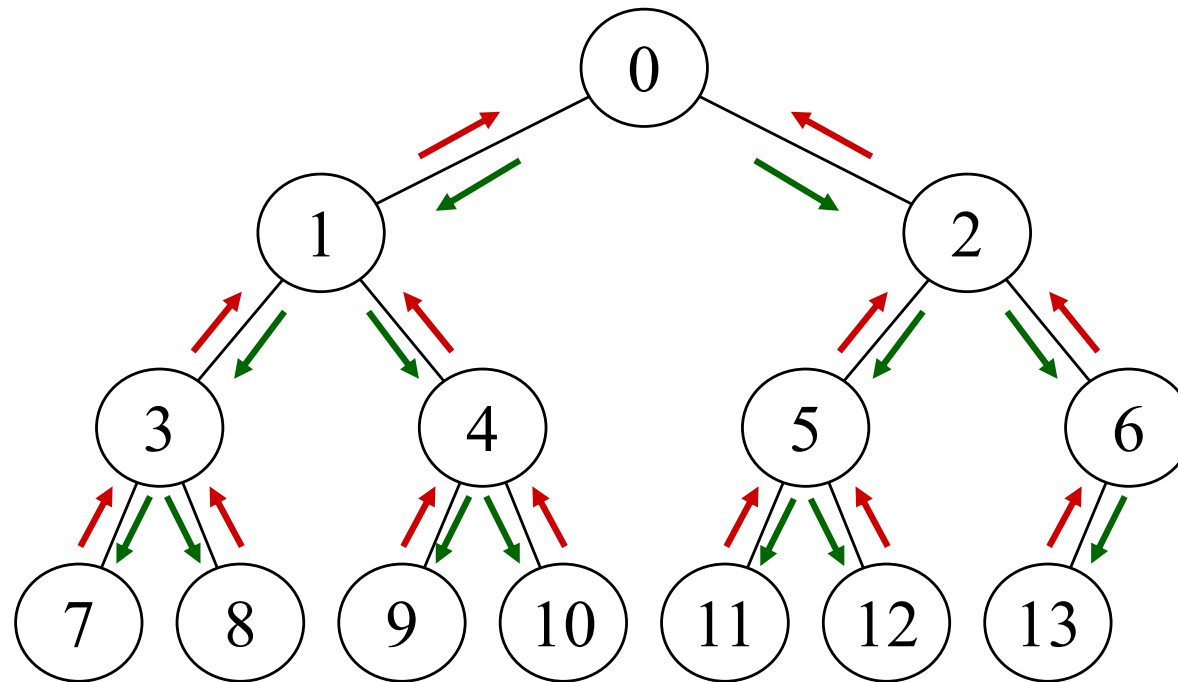
Broadcast Barrier



1 step approach

- Each process broadcasts message when it reaches barrier
- Wait until a message is received from each other process
- $N(N-1)$ messages

Tree Barrier



- Organize processes into a tree
- A process sends a message to its parent process when
 - The process has reached the barrier point, and
 - A message has been received from each of its children processes
- Root detects completion of barrier, broadcast message to release processes (e.g., send messages down tree)
- $2 \log N$ time if all processes reach barrier at same time

Outline

- Synchronous Algorithms
 - Barrier mechanisms
 - Centralized Barriers
 - Tree Barrier
- Transient Message Problem
- Performance of Conservative Approaches

A Simple Synchronous Algorithm

- N_i = time of next event in LP_i
- LA_i = lookahead of LP_i

WHILE (unprocessed events remain)

 receive messages generated in previous iteration

$LBTS = \min (N_i + LA_i)$

 process events with time stamp $\leq LBTS$

 barrier synchronization

Transient message: one that has been sent, but has not yet been received at its destination

The algorithm fails if transient message(s) remain after the processes are released from the barrier

Transient Message Problem

- All of the LPs advance to simulation time T
- One LP sends a message with **time stamp $T + 1$** , while others generate messages with **time stamp $T + 10$** .
- Assume that all of these messages are transmitted to their destination processor **except the message with time stamp $T + 1$, which is delayed in the network**.
- All processors enter the barrier primitive and begin the computation to determine the events that are safe to process. As the time stamp $T + 1$ message *has not been received*, **it is not considered in this computation**.
- The processors erroneously **consider the time stamp $T + 10$** events are all **safe** to process, since there are none with a smaller time stamp.

Flush Barrier

No process will be released from the barrier until

- All processes have reached the barrier
- **Any message sent by a process before reaching the barrier has arrived at its destination**

Revised algorithm:

WHILE (unprocessed events remain)

 receive messages generated in previous iteration

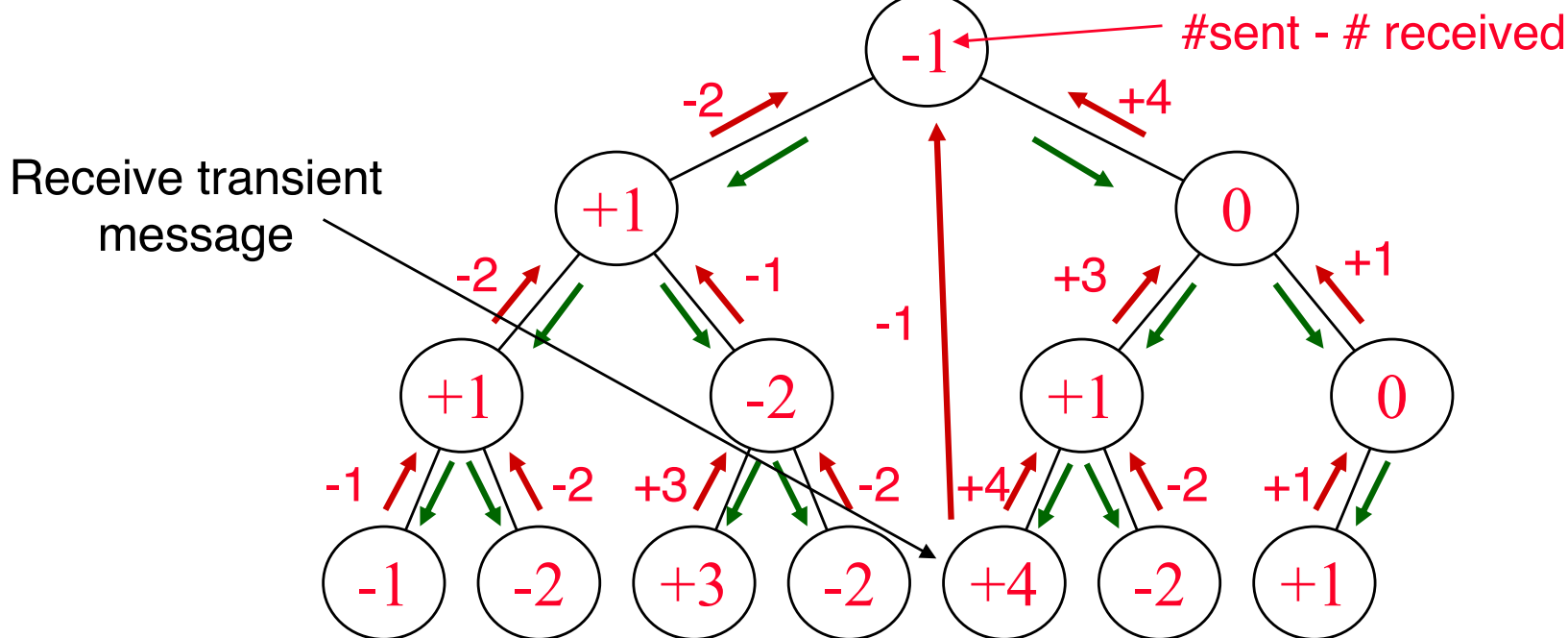
$LBTS = \min (N_i + LA_i)$

 process events with time stamp $\leq LBTS$

 flush barrier

Implementation

- Use FIFO communication channels, send a message on each channel
- Message counters
 - Send_i = number of messages sent by LP_i (this iteration)
 - Rec_i = number of messages received by LP_i (this iteration)
 - There are no transient messages when
 - All processes are blocked (i.e., at the barrier), and
 - $\sum \text{Send}_i = \sum \text{Rec}_i$



- When a leaf process reaches flush barrier, include counter (#sent - #received) in messages sent to parent
- Parent adds counters in incoming messages with its own counter, sends sum in message sent to its parent
- If sum at root is zero, broadcast “go” message, else wait until sum is equal to zero
- Receive transient message: send update message to root

Outline

- Synchronous Algorithms
 - Barrier mechanisms
 - Centralized Barriers
 - Tree Barrier
- Transient Message Problem
- Performance of Conservative Approaches

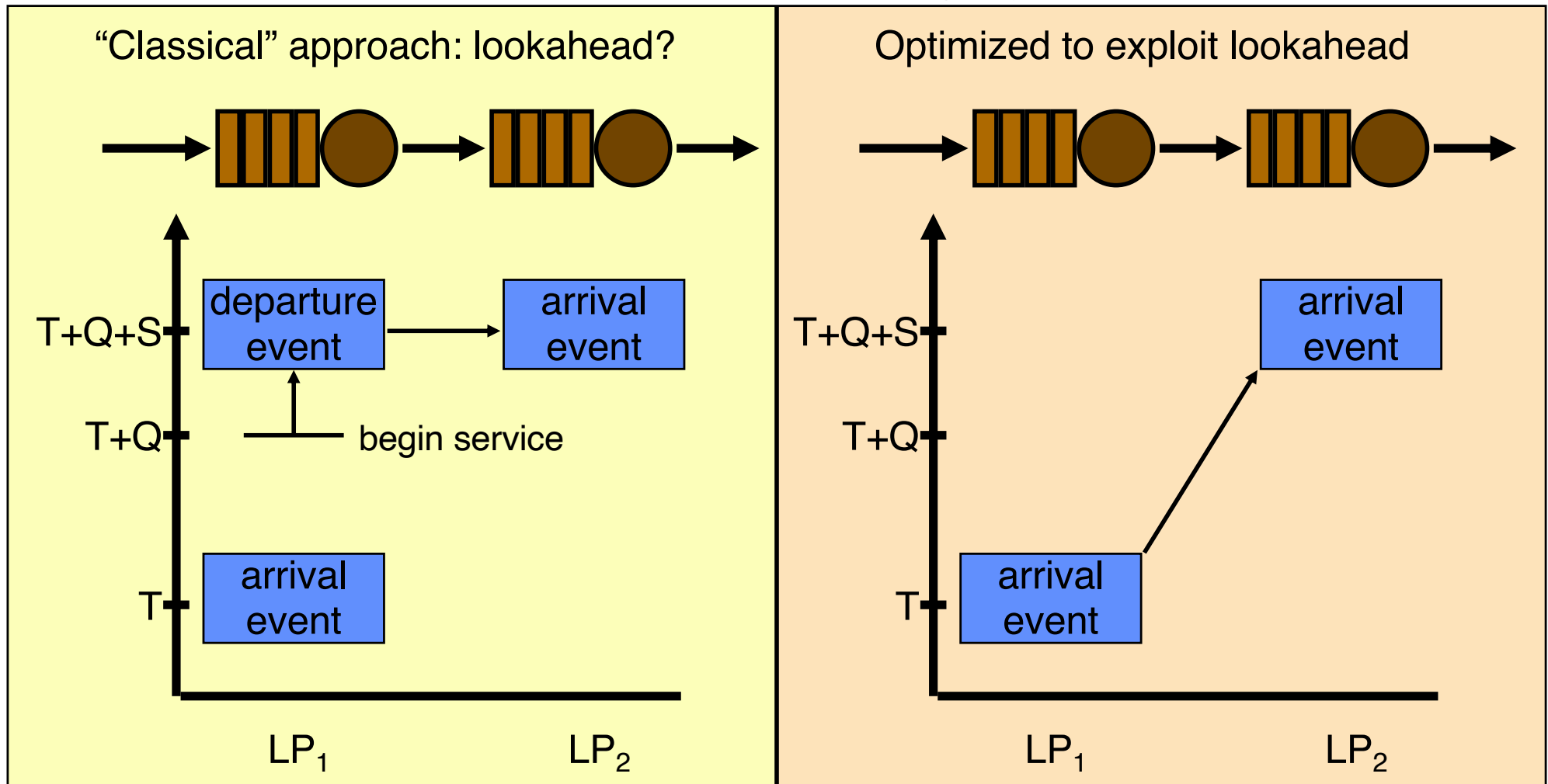
Performance

Example: Tandem first-come-first-serve queues

T = arrival time of job

Q = waiting time in queue

S = service time



Maintain variable indicating departure time of previous job

Performance

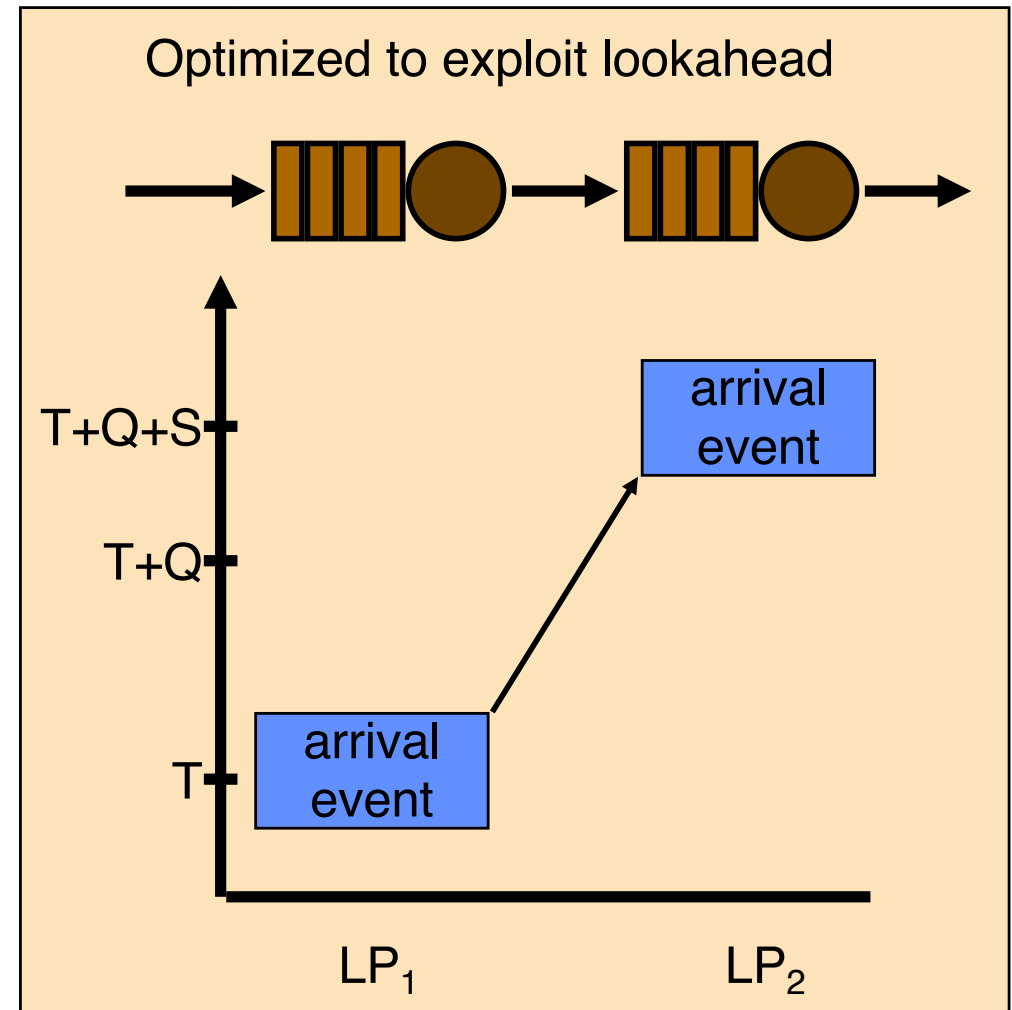
The variable **Done** is introduced to indicate when the server would have served all jobs currently in the queue.

- If arrival time $T_a > \text{Done}$
 - ✓ the server is idle when the job arrives,
 - ✓ the job completes service at time $\text{Now} + S$.
- Otherwise,
 - ✓ the server is busy until the time stored in the **Done** variable.
 - ✓ The job will depart at time $\text{Done} + S$.

Observation: one way of viewing **lookahead** is to note that the arrival event at time $T + Q + S$ is invariant to any other events occurring in the interval $[T, T + Q + S]$. This allows the event to be generated at time T

The classical simulation program has poor lookahead properties because LP1 must advance to simulation time $T + Q + S$ before it can generate the new arrival event with time stamp $T + Q + S \rightarrow$ **zero lookahead**

T = arrival time of job
 Q = waiting time in queue
 S = service time



Performance

Classical Approach (LP1)

Now = current simulation time
S = service time for job
NWait = number of jobs in queue
Busy = true if server busy

Arrival Event:

```
if (not Busy)
    Busy := true;
    schedule departure event at Now+S;
else
    NWait := NWait+1;
```

Departure Event:

```
schedule Arrival event at LP2 at Now
if (NWait>0)
    NWait := NWait-1;
    schedule departure event at Now+S;
else
    Busy := false;
```

Optimized Approach (LP1)

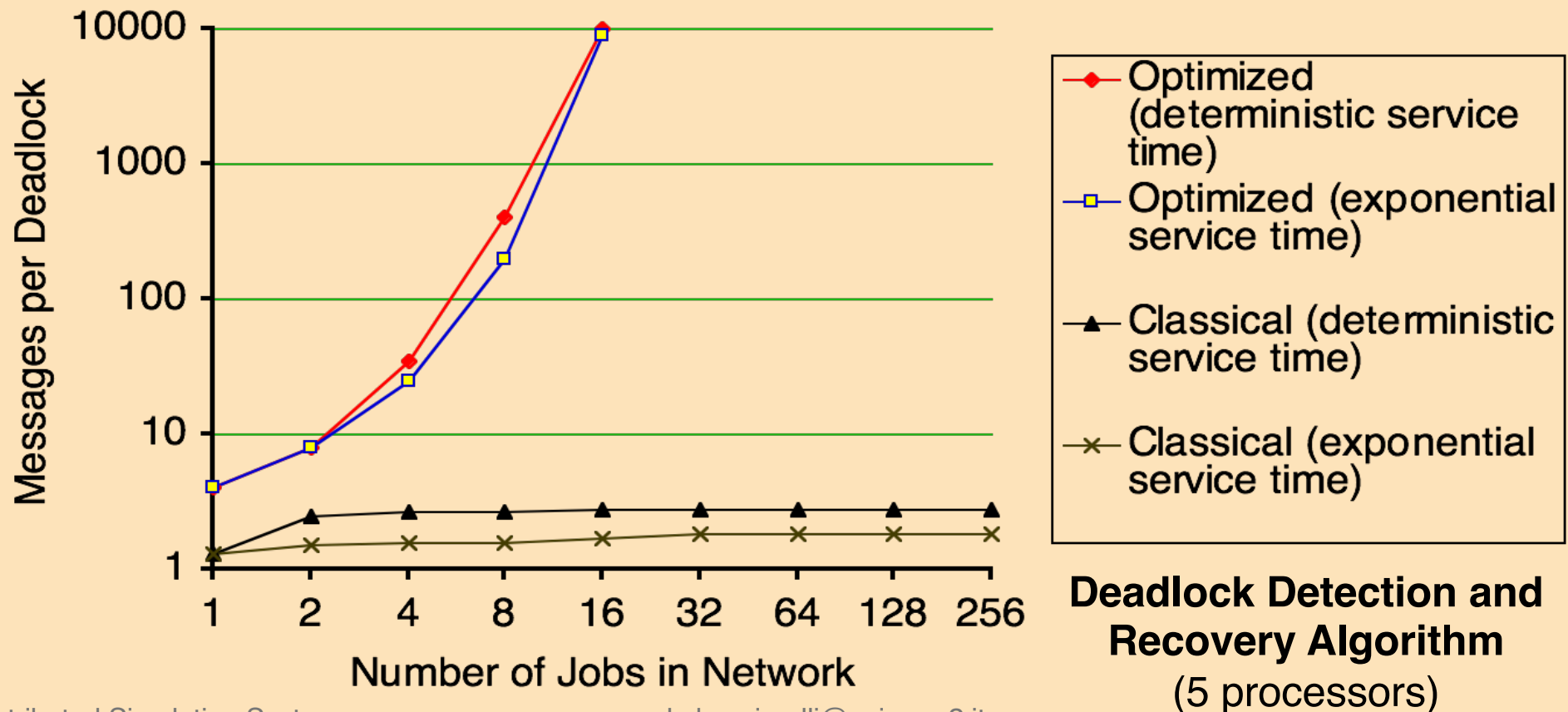
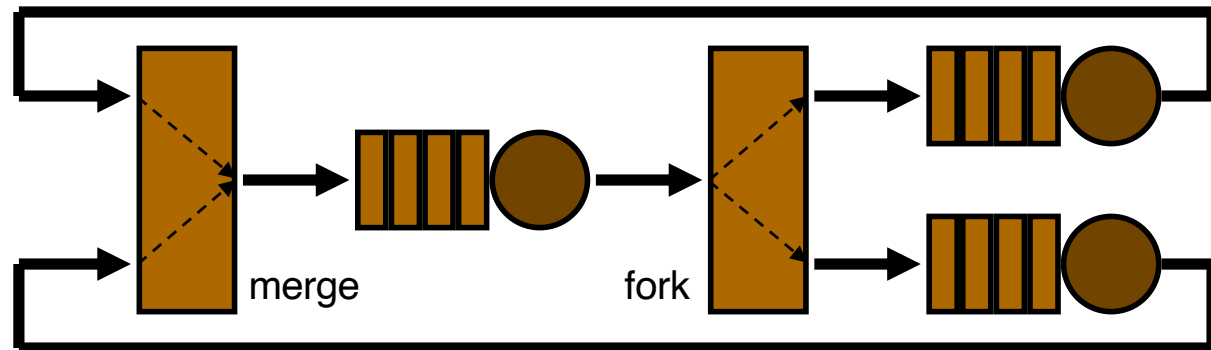
Now = current simulation time
S = service time for job
Done = time server will become idle

Arrival Event:

```
if (Now < Done)
    /* if server busy */
    Done := Done+S;
    schedule arrival event at LP2 at Done;
else
    /* server is idle */
    Done := Now+S;
    schedule arrival event at LP2 at Done;
```

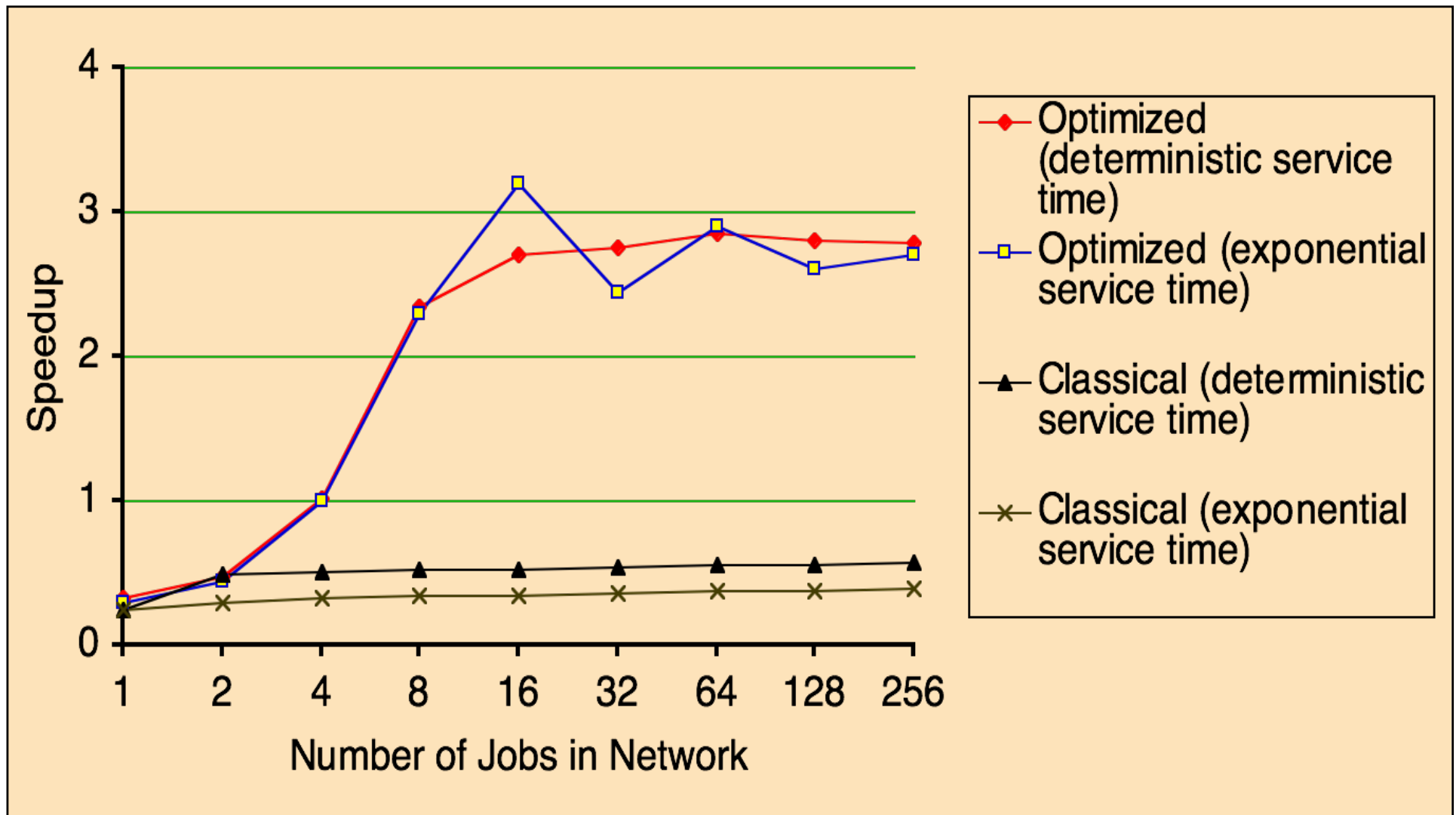
Efficiency of Queueing Network Simulation

Parallel Simulation
of a Central Server
Queueing Network



Speedup of Queueing Network Simulation

(5 processors)



Exploiting lookahead is essential to obtain good performance

Summary

- Barrier mechanisms
 - Synchronous execution approach
 - Simple barriers using broadcast or central controller
OK for small number of processors
 - Tree give more scalable performance
- Transient messages must be accounted for by the synchronization algorithm
 - Flush barrier
 - Send and receive counters
- Performance of Conservative Approaches
 - Parallel/Distributed simulation not always performs better than the sequential one!
 - Optimization is required